For Batch A

Program 1: Frequency-Based String Reconstruction

Problem:
Take a string input. Rearrange it such that:

- Characters are sorted based on decreasing frequency
- If two characters have same frequency → sort alphabetically

Example:
Input: `programming`
Output: `rrggmmpaino`

Constraints:

- Do not use `collections.Counter` or built-in frequency utilities

Program 2: Mirror Number Pattern (Palindrome Pyramid)

Problem:
Input an integer `n`. Print a pyramid pattern:

For `n = 5`

```
    1

   121

  12321

 1234321

123454321

 1234321

  12321

   121

    1
```

Constraints:

• No string slicing
• Must construct numbers using mathematical logic

• Upper and lower halves must be generated using loops (no hardcoding)

Program 3: Hidden Substring Detector

Problem:
Given two strings `s1` and `s2`, check whether `s2` exists inside `s1`.

Example:
Input:
```
s1 = "datastructure"
s2 = "struct"
```
Output: `Found at index 4`

Constraints:

- Do not use `in`, `find()`, or slicing
- Implement using manual comparison logic

Program 4: Self-Transforming List

Problem:
Take a list of integers and transform each element:

- Even number → replace with sum of digits
- Odd number → replace with product of digits
- If result is multi-digit → repeat process until single digit

Example:
Input: `[24, 37]`
Output: `[6, 2]`

Constraint:

- Must use nested loops and number manipulation

Program 5: Longest Unique Substring

Problem:
Find the length of the longest substring with all unique characters.

Example:
Input: `abcabcbb`
Output: `3` (`abc`)

Constraints:

- Do not use `set()` or advanced libraries
- Solve using basic loops and logic

For Batch C

Program 6: Matrix Boundary Rotation

Problem:
Given an `n × n` matrix, rotate only the boundary elements clockwise by one position.

Example:

```
Input:
1 2 3
4 5 6
7 8 9

Output:
4 1 2
7 5 3
8 9 6
```

Constraints:

- Do not use extra matrix
- In-place modification required

Program 7: Prime Frequency Analyzer

Problem:
Given a list of integers:

- Count frequency of each number
- Print only those numbers whose frequency is a prime number

Example:
Input: `[2,2,3,3,3,4,4,4,4]`
Output: `2 3` (frequencies 2 and 3 are prime)

Constraints:

- Write your own prime-check function

Program 8: Number Spiral Diagonal Sum

Problem:
Generate an `n × n` spiral matrix (clockwise starting from 1) and calculate the sum of both diagonals.

Example:
For `n = 3`

```
1 2 3
8 9 4
7 6 5
```

Output: `25`

Constraints:

- Avoid using predefined spiral utilities
- Logic must be built manually

Program 9: Anagram Grouping Without Sorting

Problem:
Given a list of words, group all anagrams together.

Example:
Input: `["eat", "tea", "tan", "ate", "nat", "bat"]`
Output:
```
["eat","tea","ate"]
["tan","nat"]
["bat"]
```

Constraints:

- Do not sort strings
- Compare using character frequency logic

Program 10: Recursive Digit Reduction Game

Problem:
Take an integer:

- If even → divide by 2
- If odd → multiply by 3 and add 1
  Repeat until number becomes 1.

Print total steps.

Example:
Input: `6`
Output: `8 steps`

Constraints:

- Must use recursion (no loops)